第21组项目过程管理

杜鑫（2110531）、邹佳衡（2312322）、蒲天轶（2113211）、张昌源（2313177）、宋灿灿（2312136）

- 第21组项目过程管理
  - 项目实现成果
  - 项目管理过程
  - 项目开发过程
  - 团队贡献情况
  - 项目完成情况
    - 需求完成度
    - 整体完成度
    - 目前存在的问题
    - 未来发展方向

项目实现成果

如下图是项目大概的树状图：

```
BookFlow_演示版/
├── backend/                          # Python 后端 (FastAPI)
│   ├── bookflow_core/
│   │   ├── main.py                   # 应用入口
│   │   ├── api/                      # API 路由层 (19 个模块)
│   │   │   ├── catalog.py            #   图书编目接口
│   │   │   ├── circulation.py        #   借阅流通接口
│   │   │   ├── dashboard.py          #   数据看板接口
│   │   │   ├── auth.py               #   管理员认证
│   │   │   ├── reader_auth.py        #   读者认证
│   │   │   ├── reader_portal.py      #   读者端接口
│   │   │   ├── super_admin.py        #   超管接口
│   │   │   ├── ai.py / ai_kernel.py  #   AI 接口
│   │   │   └── ...
│   │   ├── services/                 # 服务层 (40 个模块)
│   │   │   ├── catalog_service.py    #   图书业务逻辑
│   │   │   ├── circulation_*.py      #   借阅/续借/预约服务
│   │   │   ├── analytics_service.py  #   数据分析聚合
│   │   │   ├── notification_*.py     #   消息通知服务
│   │   │   ├── reader_service.py     #   读者管理
│   │   │   └── ...
│   │   ├── ai/                       # AI 内核
│   │   │   ├── kernel/               #   任务队列、工作流引擎
│   │   │   ├── chat/                 #   对话管理
│   │   │   ├── engine/               #   LLM 客户端 (DeepSeek)
│   │   │   └── tools/                #   AI 工具注册与调度
│   │   ├── schemas/                  # Pydantic 请求/响应模型
│   │   ├── db/                       # 数据库初始化与迁移
│   │   ├── core/                     # 配置、数据库会话、异常
```

```
│   │   └── ai_tools/                # AI 可调用的业务工具
│   └── data/
│       ├── bookflow.db              # SQLite 数据库 (100 本书)
│       └── uploads/                 # 静态文件 (封面、头像)
│
├── frontend/                        # 前端 (React 19 + Vite + TS)
│   ├── user/                        # 读者端 (端口 5173)
│   │   └── src/
│   │       ├── pages/               #   首页、搜书、借阅、通知、个人中心
│   │       ├── components/          #   图书卡片、AI 助手、通用组件
│   │       ├── services/            #   API 调用封装
│   │       ├── hooks/               #   TanStack Query 数据钩子
│   │       └── styles/              #   21 个 CSS 模块
│   ├── bookflow-admin/              # 管理端 (端口 5174)
│   │   └── src/
│   │       ├── pages/admin/         #   首页、编目、借阅、数据中心、任务
│   │       ├── components/          #   表格、图表、弹窗、布局
│   │       ├── api/                 #   13 个 API 模块
│   │       └── utils/               #   图表构建、PDF 导出、格式化
│   ├── bookflow_superadmin/         # 超管端 (端口 5175)
│   │   └── src/
│   │       └── pages/               #   治理总览、管理员、审计、AI 规则
│   └── common/                      # 三端共享模块
│       └── src/                     #   登录组件、HTTP 封装、工具函数
│
├── tests/                           # 测试 (10 个测试文件)
│   └── core/                        #   后端核心测试
├── scripts/                         # 运维脚本 (数据导入、校验)
├── docs/                            # UI 设计参考截图
├── artifacts/                       # 导出文件
└── README.md
```

项目采用**前后端分离**架构，后端基于 FastAPI 分为 API 路由层、Service 服务层和数据层（SQLAlchemy ORM + SQLite），AI 内核通过 DeepSeek API 以人机协同方式参与业务处理；前端为三个独立 React 19 单页应用（读者端、管理端、超管端），通过 Vite proxy 统一转发至后端 8000 端口，共享 `common/` 公共模块。项目总代码量约 **5.5 万行**（后端 Python 2.4 万行，前端 TypeScript/CSS 2.4 万行，测试与脚本 0.6 万行），数据库包含近 20 张表、100 本预置图书和完整测试数据。

项目管理过程

项目自 5 月 1 日正式启动，至 6 月 13 日完成最终交付，历时约六周。整体管理采用"原型法 + 迭代式开发"的混合模式：早期通过快速构建页面原型与用户确认需求方向，降低返工风险；中后期按三个迭代周期逐步叠加功能，每个迭代均遵循"需求分析 → 设计 → 编码 → 测试 → 评审"的闭环流程。

团队共五人，采用逐个模块开发——后端 API 与数据库、读者端前端、管理端与超管端、AI 模块集成、测试与审计等都有团队成员负责。每周进行一次进度同步，及时调整任务优先级，确保关键路径不受阻塞。

风险管理方面，项目初期识别出"AI 全程主导导致数据安全不可控""三端功能边界模糊""业务状态流转复杂易出错"等五个核心风险，逐一制定了应对措施：将 AI 降级为可控参与模式、明确三端职责划分、通过 Service 层集

中管理业务规则等，有效保障了项目按期推进。

需求变更管理方面，项目初期团队将系统定位为"AI 完全主导的图书馆业务平台"，即 AI 直接处理自然语言并操作数据库。实际开发中发现图书馆业务涉及真实数据状态和权限边界，AI 直接修改业务数据容易带来误操作和越权风险。经评审后，团队将 AI 角色从"完全主导"变更为"可控参与"——AI 仅生成分类建议、异常检测、咨询回复等辅助性结果，所有写操作须经管理员在待处理中心审核确认后执行，高风险操作强制转人工。此次变更使 AI 从潜在风险转变为可控的提效工具。

项目开发过程

整个开发过程分为六个阶段：

第一阶段（5 月上旬）：需求采集与设计。 确定项目主题为社区图书馆管理系统，明确目标用户（读者、管理员、超级管理员），梳理核心业务流程（借阅、归还、续借、预约、逾期处理），设计实体编码规则和页面概念图。

第二阶段（5 月中旬）：基础架构搭建。 完成后端 FastAPI 项目骨架、SQLite 数据库建模（近 20 张表）、三端前端 Vite + React 项目初始化，以及 common 公共模块的抽离。

第三阶段（5 月下旬）：核心功能开发。 实现图书编目、借阅流通、读者认证等基础业务闭环。后端完成 API 路由层和 Service 层的核心模块，前端完成读者端首页、搜书、借阅页面以及管理端编目和流通处理页面。

第四阶段（6 月上旬）：高级功能与 AI 集成。 加入 AI 内核模块（任务队列、工作流引擎、DeepSeek 客户端），实现 AI 辅助分类建议和异常检测的人机协同模式。

第五阶段（6 月上旬至中旬）：调试测试与体验优化。 导入数据，修复大量 bug 和交互缺陷，完成后端核心业务单元测试和集成测试（10 个测试文件），前端补充 loading/error/empty 三态覆盖，手工验证主要用户旅程。

第六阶段（6 月 13 日前）：收尾交付。 修复了少量 bug，完成 PPT 和项目文档撰写，准备课堂展示。

团队贡献情况

团队共有5人，分工贡献情况如下：

姓名	学号	主要贡献	贡献评分（百分制）
张昌源	2313177	前期设计讨论，底层架构和内核搭建，前后端编程，测试与修复bug，引入AI，一篇报告+PPT	40
邹佳衡	2312322	前期设计讨论，图书馆运行规则、工作流程和编码设计，前后端编程，测试与修复 bug，三篇报告+展示	35
宋灿灿	2312136	前期设计讨论，前后端编程，测试与修复bug	20
蒲天轶	2113211	前期设计讨论，使用与试运行	2.5

姓名	学号	主要贡献	贡献评分（百分制）
杜鑫	2110531	前期设计讨论，使用与试运行	2.5

项目完成情况

需求完成度

对照需求分析文档，系统按读者端、管理端、超管端三端划分功能，各模块完成情况如下：

一、读者端（用户自助服务）

功能模块	完成度	说明
注册与登录	100%	邮箱注册（含验证码）、读者登录、游客浏览模式、密码找回
首页与图书检索	100%	图书关键词搜索、分类浏览、热门推荐、馆藏地图
图书详情	100%	封面、作者、出版社、ISBN、分类、简介、馆藏位置、实时可借数量
借阅申请	100%	在线借阅，自动校验信用分、借阅上限和库存，借阅历史查询
预约排队	100%	无可借副本时预约加入队列，到书自动通知，支持取消预约
个人中心	100%	个人信息、当前借阅、历史记录、预约状态、信用分展示
通知中心	100%	分类筛选（借阅提醒/预约通知/系统消息/活动公告），未读/已读切换，点击查看详情弹窗，标为已读/清空已读
AI 助手	90%	自然语言图书查询、借阅规则咨询、取书码查询

二、管理端（馆员业务处理）

功能模块	完成度	说明
登录	100%	正常使用管理员账号进行登录
工作台首页	100%	今日概览（待处理任务、在借数量、逾期数量、预约数量），AI 生成运营摘要
馆藏编目	100%	新增图书、编辑元数据、上传封面图片、按中图法分类管理
副本管理	95%	维护每本书的物理副本状态（可借/借出/预约保留/维护中）和架位信息
借书处理	100%	为读者办理借书，校验信用分和借阅上限，确认后生成借阅记录

功能模块	完成度	说明
还书处理	100%	确认还书，自动判断逾期、更新信用分和副本状态
续借处理	100%	为读者办理续借（限一次），延长应还日期
预约管理	100%	查看预约队列，确认取书，处理过期预约
逾期管理	90%	逾期自动标记、信用分扣减、逾期读者排行、逾期催还通知
数据中心	100%	五个看板：运营总览（借还趋势+任务分布）、馆藏健康（状态热力图）、热门冷门分析（排行+气泡图）、读者分析（类型/信用/活跃度）、AI 效率（成功率+工具分布），支持 7/14/30 天切换和一键 PDF 导出
待处理中心	100%	展示所有 AI 生成的任务（分类建议/异常标记/读者咨询），管理员逐一审核处理
通知管理	90%	系统通知查看，到期/逾期/预约提醒自动触达

三、超管端（系统全局治理）

功能模块	完成度	说明
登录	100%	正常使用超级管理员账号进行登录
治理总览	100%	全局 KPI 仪表盘（图书总数/馆藏副本/读者总数/当前借出/今日借阅归还/预约/逾期/审计事件），近七日借还趋势图，馆藏分布圆环图，热门图书 Top 5
读者活跃概览	100%	活跃读者占比环形图，今日活跃/新增/总活跃数
管理员账号管理	90%	创建/停用管理员账号，重置密码，查看登录记录
审计日志	95%	多条件筛选（操作者/类型/对象/时间/结果），日志详情查看，权限越权拦截
AI 规则配置	85%	AI 规则启用/禁用，风险等级和优先级配置
系统健康监控	100%	数据库/规则/AI/导出四维度健康评分，系统关键活动时间线，服务状态展示

整体完成度

三端共 27 项功能模块全部达到可演示水平。读者端完整覆盖"查书→借阅/预约→取书→记录追踪"的自助服务闭环；管理端覆盖"编目→流通处理→数据看板→任务审核"的馆务全流程；超管端覆盖"全局监控→账号管理→审计

追溯→规则配置"的治理需求。AI 以人机协同方式贯穿三端，形成了"读者自助—馆员审核—超管治理—AI 辅助"的完整业务体系。

目前存在的问题

1. **仅支持本地部署运行。** 当前系统依赖本地运行，未进行容器化或云端部署，外部用户无法直接访问。
2. **AI 自主能力有限。** AI 管理员的自然语言理解范围局限于预设任务类型（图书查询、分类建议、规则咨询等），对复杂多轮对话和模糊意图的处理能力不足；AI 生成的结果仍需管理员审核确认。
3. **身份与权限体系可进一步细化。** 当前管理员和超级管理员的权限边界较为粗略，缺少基于角色的细粒度权限模板（如仅馆藏维护权限、仅流通处理权限等），普通管理员的部分操作权限依赖代码层面硬限制。
4. **流通规则缺少可视化配置界面。** 借阅上限、借阅期限、预约保留时间、信用分阈值等核心参数当前使用系统默认值，超管无法通过界面直接修改，需改动代码或数据库。
5. **测试覆盖率不足。** 当前 10 个后端测试文件覆盖了核心业务逻辑和部分集成场景，但前端组件测试和端到端自动化测试尚未建立，回归测试依赖手工验证。
6. **未接入外部通知渠道。** 系统内的到期提醒、逾期通知、预约到书通知仅支持应用内展示，未对接短信或邮件网关，读者离开系统后无法及时获取提醒。

未来发展方向

技术层面： 采用 Docker 容器化部署，降低环境依赖和启动复杂度；引入 Redis 缓存热点数据以支撑更大规模的并发查询；增加 WebSocket 实现通知消息的实时推送；建立前端组件自动化测试和端到端测试流水线。

功能层面： 完善流通规则的可视化配置界面，使超管无需改动代码即可调整借阅参数；对接外部邮件和短信服务，实现离线场景下的读者通知触达；引入推荐算法，基于读者借阅历史提供个性化图书推荐。

业务层面： 支持多分馆联盟模式下的馆际互借与图书调拨；对接公共图书馆标准的 MARC 数据格式，降低图书数据导入成本；增加 RFID 硬件对接接口，实现自助借还和智能盘点。

AI 演进方向： 当前已完成"完整业务平台 + 可控 AI 辅助"的阶段目标。后续将逐步扩展 AI 管理员的任务覆盖范围，在保持人机协同审核底线的前提下，对低风险常规任务探索自动执行模式，向更智能、更自治的阅读服务平台演进。